DS & AI - Seeing through the hype [edition 2026/2027]

Data Quality (DQ)
Lab 1: Sensitivity of ML to DQ problems

Maurice van Keulen

August 30, 2026

Introduction

Goal The goal of this lab is to experiment with data quality problems of various kinds, using various machine learning methods, under various conditions, to see what the effect is of these DQ problem on the performance of the resulting ML model.

Overview You will do this lab in groups of three (self-enrollment via Canvas). You are provided with two Python Notebooks that contain code to train and evaluate a regression task and a classification task, respectively, each for one particular data set and for one particular ML method. This is your starting point. You are given a set of *exercises*. Each pertains to an *experiment* or an *expansion* which describe possible experiments that you could try and possible ways in which you can make your experiments more interesting, respectively. **You need not do all exercises but should strategically select exercises!** The idea here is that you can choose among the exercises yourselves, for example, based on your ambitions. See below under Exercise Structure for details. Each exercise requires you to modify and extend the Python notebook. Based on your selection of experiments, you are asked to write a brief reflection. Since there would otherwise be nothing relevant to conclude, you are required to do at least one 'experiment' among the exercises.

Exercise Structure You can choose the combination of exercises yourself. There are exercises describing a particular experiment and there are exercises describing an expansion, i.e., a way to make all your experiments more interesting (hence also more complicated). Each exercise has an associated number of points. **You do not need to do all exercises but should strategically select exercises!** Choose the combination you find most interesting. Moreover, the exercises are also categorized as Easy, Regular, and Advanced with the following intention:

- If you know you are a good student and would like a challenge, then skip the easy stuff and go to the regular and/or advanced directly. Since an enhancement is not an experiment, you'd need to at least have one 'experiment' exercise in your combination (there are several easy and regular ones, hence making it more challenging for yourself means to combine one or two experiments with several expansions).
- If you know yourself to be a struggling student aiming to compensate with hard work, then go for a sufficient number of easy exercises.
- If you don't have the ambition of achieving a high grade here, then you can limit the work to a few exercises aiming for a 6. Do allow for some margin in this case, because you may not get the full points for the exercises or the reflection.

Reflection report Write a 1–2 page reflection report (body text; figures, tables, and captions may be included in addition without a fixed limit) on your experiments investigating the impact of data quality problems on machine learning performance. Include the goal the experiments and what you expected to find. Compare the effects of different types of data quality issues (e.g., noise, missing data, limited training data) on model performance, and discuss any patterns or surprising findings across datasets or ML methods, if applicable. Support your conclusions with evidence from your experiments, including references to specific graphs or metrics, and note any results that were unexpected or difficult to explain. Only include figures or tables that are directly discussed in the text. Consider the practical implications: when might these issues be most problematic in real-world applications, what strategies might practitioners use to mitigate them, and what are the limitations of your experimental setup for drawing these conclusions?

The reflection report will be assessed on three criteria: (1) Understanding of Experimental Goals, (2) Understanding of Experimental Results, and (3) Depth of Reflection on the Results.

Points distribution The grade for this lab is established in the following manner:

Exercises	sum of exercise points max 20 points
Reflection report	max 10 points
Total	max 30 points
Minimum	There is no minimum requirement No submission means 0 points

Just to help you with planning which exercises to do, here is the reasoning I used for determining the points. For a 6.0, you'd need to have 18 points. A just sufficient score for the reflection report is 6 points, so for the exercises you'd need 12 points. Two exercises (two experiments or one experiment and one expansion) will already give you that, but it is too risky to leave it at that: you might not get full points for these exercises. Therefore, it would be wise to do at least three exercises.

Deadline, deliverables and mode of delivery The intention is that you can finish the lab during the scheduled 4-hour lab session. For reasons of flexibility and resiliency to exceptions, the deadline is one week later. You need to submit the deliverables listed below to Canvas in the indicated formats. One submission per group.

- For each exercise, submit a separate PYNB file of the Python Notebook you developed to execute the experiment. Use the following naming convention: "`Group-`N`-DSAI-DQ-Lab-1-Exercise-`M`.pynb`" where N is your group number and M is the number of the exercise. It is okay (actually the intention) to make a copy of your Python Notebook of a completed exercise when you start a new exercise and continue in the copy.
- A separate PDF file named "`Group-`N`-DSAI-DQ-Lab-1-Reflection.pdf`" with the reflection report (around 1 or 2 pages A4).
- Don't forget to put your names, student numbers, and group number in each of these files.

Data sets The lab comes with a collection of data sets (`Datasets.zip`). For your convenience, we already divided them into 'small' and 'large', i.e., whether they contain more than 500 rows. The given two Python Notebooks use the following two files, respectively:

- `DSAI-DQ-lab1-Classification.ipynb`: Uses "`credit-a.arff`" which is in the `uci-large` folder.
- `DSAI-DQ-lab1-Regression.ipynb`: Uses "`stock.arff`" which is in the `regression-large` folder.

The other files are there to choose from for exercises that expand the number of data sets.

NB: The `.arff` format is a textual format like CSV (just inspect it with an editor). It is a format defined by the developers of the WEKA toolkit. It also has the data as one line per row of data. But ARFF has more documentation about the data in it. So, do inspect the ARFF file with an editor to read the documentation of the attributes and other information. The Python Notebooks contain a function with which you can read ARFF files.

Lab environment The indended environment for doing this lab, is simply Python with Python Notebooks on your own laptop. An alternative is UT's Jupyter Lab (`https://jupyter.utwente.nl`). Documentation can be found in the UT-JupyterLab wiki (`https://jupyter.wiki.utwente.nl/`) or inside Jupyter Lab: Menu 'Application' → 'Jupyter Wiki'). See below under "Do's and Don'ts" for non-intended but allowed alternatives.

Do's and Don'ts Although we prepared everything for Python, this is only one means to an end. You are also allowed to do this lab in R or MatLab or with some Data Science / Analytics tool of your choosing. Note that in this case, you will need to start from scratch, because we only prepared Python Notebooks as a kick-start for the lab. If you go for something else than Python, submit the source file(s) of your code as replacement for the PYNB deliverable. If you choose to go for an alternative programming language, then we appreciate similar kick start source files with a single regression and classification task in it similar to the given Python Notebooks. You would have to develop it anyway, and in this way, we may provide it to future students.

You need not use the scheduled lab session for this lab. You could do it later by yourself without any help from teaching assistants. You would need to do it within one week of the schedule lab session to meet the deadline.

Above, it says groups of 3 persons. You are allowed to work in pairs or even alone, but we prefer you to work in groups of 3. Note also that there is no reduction in workload if you choose to work in a smaller group.

Permission to make use of Generative AI In this lab, you may use Generative AI to write and improve your code for the exercises, since coding proficiency is not a major learning objective here; the goal is to build awareness of the effects of data quality problems on machine learning performance by experimenting with them yourself. Still, use this as a learning opportunity: ask the AI to explain any code you find unfamiliar or strange, since one of generative AI's most valuable uses today is as a personal teacher.

For writing the reflection report, you may use Generative AI to improve your English and formulation. The main goals of the lab are raising awareness and developing your reflection skills, not improving your writing skills. So you may give the AI your main conclusions and reflections and ask it to improve the English, formulation, or conciseness if it becomes too long. However, the analysis, interpretation, and reflection on the results must be your own work.

Global assessment of DQ part There are two labs for the DQ part. Lab 1 (this one) amounts to 30 points and lab 2 amounts to 70 points. The sum of the points of both labs determines the DQ lab grade. This grade has weight 20% for the final grade of the DQ part.

$$\textit{DQ Lab Grade} = \frac{1}{10}(\textit{Points lab 1} + \textit{Points lab 2}) \quad (1)$$

$$\textit{Final DQ grade} = \frac{1}{5}(\textit{DQ Lab Grade}) + \frac{4}{5}(\textit{DQ Test Grade}) \quad (2)$$

There is a minimum requirement of a 5.0 on the DQ lab grade. Since the lab grade is based on both labs, the one lab could compensate for a bad performance on the other lab. There is a repair, but it is for both labs together and you can only achieve a maximum of 60 points for the repair (i.e., a DQ lab grade of 6.0).

Exercises

Classification (predicting a class label) is often easier than regression (predicting a numerical value). Therefore, the easy experiments begin with classification only, hence will only use one of the two given Python Notebooks. This implicitly also means we are focussing on the file `credit-a.arff` where the goal is to classify whether a credit will be approved or not based on several input features, and a Naive Bayes classifier as ML method. If you like to challenge yourself, you can focus on regression or do both; see Exercise 8.
NB: There are comments in the Python Notebooks to help you understand the steps it takes. If it uses terms that are unfamiliar to you (such as "one-hot encoding"), then do look them up to understand what they mean.

1 [Easy] Experiment: "Effect of random noise in input features on classification"

Points: 6 points for a correctly carried out experiment

Extend your program such that it adds several different amounts of noise to one of the input features and trains and evalutes the model again, so that you can compare the evaluation results to see how it changes with these different levels of noise. We add the noise in the following manner:

- Keep the original unaffected train and evaluation code untouched in your Python Notebook, because you need to compare results. First, record the model's performance on the original, unmodified dataset as your baseline.
- Choose one attribute as the focus of your corruption with noise. If it is numerical, determine its minimum and maximum over all rows. If it is categorical, determine a list of all possible values for the attribute.
- Loop over several amounts of noise, for example, 5%, 10%, 20%, 50%. You may need to finetune this list, so that you can see a nice trend in your result.
- For each noise level (e.g., 5%), randomly select that percentage of ALL rows and replace the chosen attribute's value in those rows only. Use different randomly selected rows for each noise level experiment. If the chosen attribute is numerical, then keep the random number within the minimum and maximum. If it is categorical, choose the random value from the list of possible values.
- Train and evaluate the model as normal to determine the effect of the noise on the classification model's performance.
- Show the evaluation results in a graph in the notebook. Your graph should clearly show the original performance as the 0% noise point and compare all noise levels against this baseline to show performance degradation.

Table 1: Guessed (unofficial) meanings of attributes in the UCI Credit Approval dataset circulated in Kaggle/tutorial sources.

Attribute	**Guessed meaning**
A1	Gender
A2	Age
A3	Debt
A4	Marital status
A5	Bank customer
A6	Education level
A7	Ethnicity
A8	Years employed
A9	Prior default
A10	Employed
A11	Credit score
A12	Driver's license
A13	Citizen
A14	ZIP code
A15	Income
A16	Approval status (class)

2 [Easy] Experiment: "Effect of random noise in class labels on classification"

Points: 6 points for a correctly carried out experiment

This experiment is analogous to the previous one, but instead of choosing an input feature (attribute), we here focus on the class label (class attribute). Apply noise only to the class labels in the TRAINING data. Never corrupt the class labels in your test/evaluation data, as this would make it impossible to properly measure model performance. The test set should remain clean to provide reliable performance metrics.

3 [Easy] Experiment: "Bias in the data"

Points: 4 points for a correctly carried out experiment;
2 points for repeating the experiment for an attribute with more than two possible values (e.g., A4, A5, or A13)

It is said that "bias in the data created bias in the model". Analogous to the previous experiments, we're going to see how strong that effect is. The dataset `credit-a.arff` has been anonymised, but in table 1 we provide guessed meanings for the attributes. The idea is that we train models with increasing bias in some attribute (e.g., gender)

and then evaluate the performance on a balanced test set as well as on the categories of the biased attribute separately. Proceed in the following manner:

- Take the original workbook for the classification experiment (i.e., without the noise corruption code of the previous two experiments). We choose attribute A1 (presumed to be 'gender').
- Split the dataset into two parts A and B with only the rows A1=a and A1=b, respectively. Construct two test sets by taking 20% of each leaving 80% of each for training.
- Loop over several ratios of bias, say 0%, 5%, 10%, 20%, 30%, 40%, 50%. Take a basis training set to be the one with only the training rows left for A. Then replace the ratio percentage of randomly chosen rows by randomly chosen rows from B.
- Train a classifier for the given dataset and measure its performance for the A test set, B test set, and for the combined test set.
- Show the evaluation results in a graph in the notebook.

4 [Easy] Expansion: "Multiple data sets"

Points:

1 point for each additional data set from the collection used in your experiments upto a maximum of 4 data sets; 2 points for each additional data set that you found yourself, used in your experiments upto a maximum of 3 data sets found by yourself; 2 points for a methodically correct expansion of the experiments to a multi-dataset setting; Max 8 points for this expansion

Obviously, doing experiments on only one data set does not say much in general. To be able to draw more general conclusions, it is better to do the experiments on several data sets and aggregate the evaluation results.

In more detail, choose at least four additional data sets, so that you have at least five in total. They may come from the given ones or you may go on the internet and search for others (worth more points). The data sets we a looking for need to comply with the following criteria:

- They are desired to have more than 500 rows. You can use smaller ones, but then you may need to formulate your conclusions more carefully in the reflection. In the set of given data sets, the data sets in the "small" folders have less than 500 rows.
- They should have several numerical and/or categorical attributes. We cannot use string-valued attributes here. They may be there, but you should remove them from the training data.

- There should be a classification task, i.e., some attribute that you can use as a class label such that you can train a model predicting the class label given the other attributes as input features.

Use all data sets for all your experiments. For each data set, train and evaluate as normal. When aggregating results across datasets, report both individual dataset results and summary statistics (mean, standard deviation, min/max) for each performance metric. Create comparison tables or graphs showing how each dataset responds to the same data quality problem. Discuss which datasets are most/least sensitive to each type of data quality issue and why this might be the case based on dataset characteristics (size, number of features, class balance, etc.).

5 [Easy] Experiment: "Availability of data"

Points: 6 points for a correctly carried out experiment

Often there is not sufficient data available and then one wonders whether or not it suffices. Or one still has to collect the data, which may be an expensive thing to do. Therefore, one wonders how much data is sufficient to collect. Although there is no answer to these questions in general, it could be insightful to investigate how the performance of the model is affected by the amount of training data available.

We do the experiment in the following manner:

- Use the same train/test split as in your baseline experiment. Keep the exact same test set for evaluation across all experiments to ensure that performance differences are due to training data availability, not different test sets.
- Iterate over a list of percentages of data, for example $1\%, 2\%, 5\%, 10\%, 20\%, 50\%$. You may need to finetune this list, so that you can see a nice trend in your result. The percentages refer to portions of your original training set.
- For each percentage of data, randomly select that percentage of rows from your original training set and train the model with only the selected rows. For example, if your original training set has 1000 rows, then 5% means training with 50 randomly selected rows from those 1000 training examples. Document the actual number of training examples used for each percentage to make your results reproducible.
- Train and evaluate the model as normal to determine the effect of the lack of training data on the classification model's performance.

6 [Easy] Experiment: "Missing data"

Points: 6 points for a correctly carried out experiment

In practice, one often encounters data for which not all attributes have been filled in. Most ML methods are not able to train a model when there are values missing. As will be explained in DQ lectures 2 and 3, there are many approaches to dealing with this "missing data" problem. In this exercise, you will experiment with two simple often-used ones: 'list-wise deletion' and 'mean imputation': deleting all rows that contain missing data, and filling in the missing data with the mean of the attribute, respectively.

Extend your program such that it removes several different amounts of values in the input features. Focus on numerical and categorical features only. Then apply both approaches, train and evaluate the model again, so that you can compare the evaluation results to see how it changes with these different levels of missing data and between the two approaches. You can do this experiment in the following manner:

- Choose one or more numerical or categorical attributes as the focus of your missing data corruption. Determine for these attributes the mean value of the numerical attributes and the most frequently occurring value for the categorical attributes (only in the training set).
- Loop over several amounts of missing data, for example, 1%, 5%, 10%, 20%. You may need to finetune this list, so that you can see a nice trend in your result.
- For each amount of missing data, if you choose multiple attributes for missing data corruption, apply the same percentage of missing values to EACH chosen attribute independently. For example, if you choose 2 attributes and set 10% missing data, then 10% of rows should have missing values in the first attribute AND 10% of rows should have missing values in the second attribute (these can be different rows). Delete the values by setting them to 'None'.
- *List-wise deletion*: delete all rows that have one or more missing values.
- *Imputation*: Use appropriate imputation methods for each data type: mean imputation for numerical attributes (fill missing values with the attribute's mean) and mode imputation for categorical attributes (fill missing values with the most frequently occurring value). Both methods should use statistics calculated only from the training set, not including the test set.
- Train and evaluate the model as normal with the resulting training set to determine the effect of the missing data cleaning approaches on the classification model's performance.
- Show the evaluation results in a graph in the notebook.

7 [Regular] Expansion: "k-Fold Cross Validation"

Points: 4 points for a methodically correct expansion of the experiments to a k-fold cross validation set-up

Having one fixed training and test set poses a risk: especially hard cases may happen be in the test set or vice versa, for example. A better technique is *k-fold cross validation*, which trains and evaluates a model k times, each with a different choice of data for the train and test sets.

This expansion is about replacing the fixed-train-and-test-set-approach by k-fold cross validation for all your experiments. It is common to choose $k = 5$ or $k = 10$. For each fold in the k-fold cross-validation, apply your data quality corruption (noise, missing data, etc.) only to the training portion of that fold, never to the test portion. This means you'll be creating corrupted data k times - once for each training set. The corruption should be applied independently for each fold to avoid data leakage between training and validation sets.

The technique is quite well explained on Wikipedia: `https://en.wikipedia.org/wiki/Cross-validation_(statistics)#k-fold_cross-validation`. Here is the documentation how to do it in Python: `https://scikit-learn.org/stable/modules/cross_validation.html`

Since you are evaluating the performance of the model multiple times with different test sets, you will get for each performance metric multiple values. I would like to see not only an average in the evaluation graph, but also a minimum and maximum value for each metric, for example, with an error bar around the average value. In the reflection report, when comparing two experiments, if their min-max ranges (error bars) overlap significantly, this suggests the performance difference may not be statistically meaningful - the two approaches might perform similarly. If ranges don't overlap, this indicates a more reliable performance difference. Consider the size of the overlap relative to the performance difference when drawing conclusions about which approach is truly better.

This is an expansion, i.e., it applies to all experiments that you are doing, so make sure that all evaluation graphs of all experiments above and below show error bars.

8 [Regular] Expansion: "Effects of DQ problems on regression"

Points: 8 points for an experiment that focuses on regression instead of classification.

The above experiments are described for classification. You could also focus on *regression*, i.e., predicting a continuous value. There is an example Python Notebook available for you that trains and evaluates a regression model: predicting the stock price for one company based on the stock prices of 9 other companies.[1] The given Python Notebook uses the ML method "linear regression" using the data set `stock.arff`.

This expansion is possibly in addition to classification, i.e., you can choose to do the experiments for only regression (this one) or for both regression and classification (i.e.,

[1] Perhaps not a very relevant thing to do, but there are general economic trends that push stock prices up and down together, which is a pattern a machine learning model could probably pick up and use.

doing Experiment 1 as well). If you choose to focus only on regression, you still need to complete at least one 'experiment' type exercise (not just this expansion). You can adapt any of the previous experiment exercises (random noise, missing data, etc.) to work with the regression notebook instead of the classification notebook. If you choose to do both regression and classification, you'll be implementing experiments for both tasks, which significantly increases your workload but also your potential points. Note that this expansion applies to all the experiments that you have chosen, i.e., you'd need to implement each of the chosen experiments for the regression task (or for both the classification and regression tasks if you choose to do both).

When working with regression tasks, use appropriate regression performance metrics such as Mean Squared Error (MSE), Root Mean Squared Error (RMSE), or R-squared (coefficient of determination). Do not use classification metrics like accuracy or precision. Your graphs should show how these regression metrics change with different data quality problems, and your reflection should interpret what these metric changes mean for prediction quality.

9 [Regular] Experiment: "Effect of non-random noise in input features"

Points: 6 points for a correctly carried out experiment

In Exercise 1, we introduced *random* noise to the data. But in real-world data, the imperfections in the data are often not random at all. For example, in a surgery data set, quality of the attributes for urgent surgeries is possibly worse than for surgeries planned in advance. Assuming we have a categorical 'urgency' attribute indicating 'urgent' vs. 'planned in advance' in the data set as well, then the noise depends on the 'urgency' attribute. In this exercise, we are going to experiment with non-random noise, i.e., with only introducing noise in attributes depending on the value of some other attribute.

We do the experiment in the following manner:

- First, choose one attribute (let's call it the 'selector attribute') and one of its values to define which rows will receive noise - these are your 'noise target rows. Alternatively, you could choose a numerical attribute and pick a threshold Then, for corruption, choose a different attribute (the 'target attribute') where you will actually introduce the noise. Never corrupt the selector attribute itself, since that would change which rows qualify as noise targets.

- Follow the same procedure as in Exercise 1, but now introduce the noise only in the noise target rows. For each noise level, corrupt the target attribute only in the noise target rows. When applying non-random noise, the percentage refers to the portion of noise target rows that get corrupted, not the overall dataset. For example, if 30% of your dataset consists of 'urgent' cases (noise target rows), and you want 10% noise, then corrupt 10% of those urgent cases only. Document both

the size of your noise target group and the actual number of corrupted values to make comparisons with random noise meaningful.

- Train and evaluate the model as normal to determine the effect of the non-random noise on the model's performance.

Obviously, it may be quite interesting to combine this experiment with the experiment of Exercise 1, so that you can compare the effect of random and non-random noise.

10 [Regular/Advanced] Expansion: "Introduce other ML methods"

Points:

2 points for each additional ML method used in your experiments; 1 extra point for including deep learning among the ML methods; We define 'deep learning' as training a neural network with two or more hidden layers; any other ML method is defined as 'traditional'[2] Max 6 points for the additional ML methods. 2 points for a methodically correct expansion of the experiments to a multi-ML-method setting

There are of course other ML methods around besides Naive Bayes for classification and Linear Regression for regression. I make a distinction between *deep learning* and traditional ML methods (see above). Examples of traditional methods are Support Vector Machines (SVM), Decision Trees, Logistic Regression, Regression Trees, etc. This expansion exercise is about adding other ML methods to your classification and/or regression experiments, so that you can determine whether one ML method is more robust against DQ problems then another, or perhaps more robust against noise in the input features but less robust against noise in the class labels or regression target. For deep learning implementation, a simple feedforward neural network with 2-3 hidden layers is sufficient to earn the deep learning point. Focus on comparing robustness to data quality issues rather than optimizing architecture complexity. You may use standard libraries like TensorFlow/Keras or PyTorch. Document your network architecture (number of layers, neurons per layer, activation functions) in your results.

This is an expansion, i.e., it applies to all experiments that you are doing, so make sure that all evaluation graphs of all experiments above and below show the performance of the other ML methods. If using 2-3 ML methods, show all methods as different lines/colors on the same graph for easy comparison. If using more than 3 methods or if graphs become cluttered, create separate subplots for each method but arrange them in

[2]The reason for defining a neural network with one hidden layer (i.e., an input layer, a hidden layer and an output layer) as traditional is because this kind of ML method has existed for a very long time already. It is often referred to as "artificial neural network (ANN)". Only around twelve years ago, researchers found out how to train a NN with more than one hidden layer. Therefore, this was called "deep learning" and caused a break-through in machine learning, because the performance of the resulting models was significantly higher than traditionally trained ones especially for computer vision tasks.

a single figure for comparison. Always include a summary table comparing the baseline (0% corruption) performance of all methods to establish their relative starting points before data quality degradation. If you make multiple pictures, please make a 'collage picture' of them in one file, because you can only upload one picture in the submission form for the results per experiment.

11 [Advanced] Experiment: "Hyperparameter tuning"

Points:

6 points for a methodically correct expansion of one chosen experiment to a set-up with tuning of one hyperparameter. 2 more points (so 8 points in total) for a methodically correct expansion of the experiment to a set-up with tuning of two or more hyperparameters.

A *hyperparameter* is a parameter that influences the training of the model. Almost all ML methods have hyperparameters, but you could be unaware of that, because mostly they have default values. Linear regression doesn't have hyperparameters, but variants 'ridge' and 'lasso' have a regularization parameter. Naive Bayes has an 'alpha' hyperparameter controlling Additive/Laplace smoothing. Decision trees have 'max_depth' and 'min_samples_split', For deep learning, the number of layers, sizes of the layers, other architectural changes, the loss function, and the activation function, can all be seen as hyperparameters.

The default value may of course not be the optimal value. This experiment is about selection one of your experiments above and adding a hyperparameter tuning phase to this experiment. This involves having a third separate data set: the 'validation set'. You need three separate data splits: training set (for model training), validation set (for hyperparameter selection), and test set (for final evaluation). Split your original training data into two parts: use 70-80% for training and 20-30% as your validation set. Keep your original test set completely untouched until the very end. Train models on the training portion, evaluate different hyperparameter settings on the validation set, then only after selecting the best hyperparameters, train a final model and evaluate it on the test set. This procedure is called a *"grid search"*. Limit your grid search to 2-3 hyperparameters with 3-5 values each to keep computation manageable. For example, if tuning a Decision Tree, you might test 3 values of max_depth and 3 values of min_samples_split (9 combinations total). For deep learning, limit to basic hyperparameters like learning rate and number of neurons per layer. Document your hyperparameter search space and the total number of models trained.

NB: Training a model is already quite computationally expensive. Training deep learning models even more: the more connections in your neural network, the more weights, the larger the model, the more expensive to train. With hyperparameter tuning, you train multiple models. You will notice this in how long you have to wait until your model training finishes, but more importantly, it is also indicative for how much energy is consumed. Training Large Language Models (LLMs) which are also at the core of nowadays' Generative AIs, are so large, that training them has significant environmental

influence. Just so you know and remember whenever you use a Generative AI such as ChatGPT. And don't hesitate to inform other people about the environmental influence when you hear them talking about using Generative AI, be they computer scientists themselves or non-technical users.